

前端开发规范

一、规范目的

1.1 概述

- 提高团队协作效率
- 便于前端开发以及后期优化维护
- 方便新进的成员快速上手
- 输出高质量的代码

本规范文档一经确认，前端开发人员必须按本文档规范进行前台页面开发。

本文档如有不对或者不合适地方请及时提出，经讨论决定后可以更新此文档。

二、基本准则

2.1 基本准则

- 符合 web 标准, 语义化 html, 结构表现行为分离, 兼容性优良。
- 代码要求简洁明了有序, 尽可能的减小服务器负载, 保证最快的解析速度。
- 开发时需要遵循如上基本准则，特殊情况可以有所宽限，如一些老项目的页面改造。

三、规范内容

3.1 文件命名

- 使得自己和工作组的每个成员能够方便的理解每个文件的意义
- 当我们在文件夹中使用“按文称排列”的命令时，同一种大类的文件能够在一起，以便我们查找、修改、替换
- 文件名称中不包含汉字、特殊字符、空格
- 图片命名可使用下划线(_)连接多个字符。如：menu_toolbar.png, combo_arrow.png
- CSP文件命名可以使用点号(.)连接多个字符。如：websys.chartbook.hisui.csp, websys.menugroup.csp

3.2 文件存放位置

文件名	说明
scripts	存放js脚本文件
scripts_lib	存放前端公开类库文件
html	存放html文件
images	存放图片文件

3.3 HTML开发规范

1. 单独前后端分离开发时，编码使用utf-8
2. 使用注释来说明开闭标签

```
<div>
  <!--上部查询区域开始-->
  <div class='my-form-body'>

  </div>
  <!--上部查询区域结束-->
  <!--中部区域开始-->
  <div class="my-body">
    <table>
      <tr><td></td></tr>
      <tr><td></td></tr>
      <tr><td></td></tr>
    </table>
  </div>
  <!--中部区域结束-->
</div>
```

3.4 JS开发规范

1. 向后台发数据时，应把<>"转换成&号，再发送到后台

```
function escapeHTML(e) {
  if ('string'!=typeof e) return e; return e.replace(/</g,
"&lt;").replace(/>/g, "&gt;").replace(/ /g, "&nbsp;").replace(/"/g,
"&#34;").replace(/'/g, "&#39;");
}
// 转义
var val = escapeHTML(document.getElementById('content'));
// 然后再发送给后台
```

2. 使用空格代替tab,使用2个或4个空格实现缩进
3. 每个语句必须以分号结尾，不能省略
4. 暂时不使用ES6语法
5. 不推荐代码水平对齐

```
//bad
{
  tiny: 42,
  longer:435,
  ...
};
//good
{
  tiny:42,
  longer:432,
  ...
}
```

6. 尽量少的使用eval语句

7. 常量的命名规范

- 全局常量统一放到文件最开始定义
- 常量命名应该使用全大写格式，并用下划线分割

```
//// bad
var times = 5;
//good
var TIMES = 5;
```

- 变量使用驼峰命名法且首字母小写

```
// bad
var oldepisodeid = 1;
// good
var oldEpisodeId = 1 ;
```

8. 使用字面量方式创建对象

```
// bad
var item = new Object();
var arr = new Array();
// good
var item = {};
var arr = [];
```

9. 变量命名时不要使用js关键词

name,arguments,date,title

```
name = 'my name';    // 与window.name冲突,错误命名
title = 'my title';  // 与window.title冲突, 错误命名
```

10. 不去修改参数对象属性值，不要对参数重新赋值

```
// bad
function f1(a){
  a = 1;
}
function f2(a){
  if (!a) { a = 1;}
}
// good
function f3(a){
  const b = a || 1;
}
```

11. 注释

- 使用 `/*...*/` 注释多行
- 使用 `//` 注释单选，放在代码上面一行
- 使用 `FIXME` 或者 `TODO` 帮助开发者明白问题

```
/**
 * 通过tag获得新的对象
 * @param {String} tag
 * @return {Element} element
 */
function make(tag){
  // ...
  // TODO: 增加配置解决
  let opt = {timeout:3000};
  // FIXME: 不应该使用全局变量,后期修复
  let ele = document.getElementById("id"+total);
  return ele;
}
```

3.5 CSS开发规范

- 类名定义

多个单词间使用-分割

全部使用小写字母

```
.datagrid-footer-inner,.datagrid-header,.datagrid-pager,.datagrid-toolbar {
  border-color: #ddd
}
.panel-body-noheader .layout-expand-title,.panel-body-noheader .panel-icon {
  top: 5px
}
```

3.6 HISUI开发规范

- 同一系统中只存一份HISUI库，以便统一管理
- 在HEAD标签中引入

```
<link rel="stylesheet" type="text/css" href="hisui/dist/css/hisui.css">
<script type="text/javascript" src="hisui/dist/js/jquery-1.11.3.min.js">
</script>
<script type="text/javascript" src="hisui/dist/js/jquery.hisui.js"></script>
<!--中文显示组件，英文引入hisui-lang-en.js-->
<script type="text/javascript" src="../../dist/js/locale/hisui-lang-zh_CN.js"></script>
```

- 界面加载完成调用某逻辑

```
var init = function(){
    // window loaded
    // do something
}
$(init);
```

- HISUI解析完成调用某逻辑

```
var init = function(context){
    // 第一次context为undefined,不为空跳出.
    if (!context) return ;
    $("#Loading").fadeOut("fast");
    // do something
}
$.parser.onComplete = init;
```

- 为了代码兼容性，不去直接操作HISUI生成的DOM树

3.7 iMedical前端开发规范

3.7.1 CSP开发规范

3.7.1.1 文件开头统一增加登录判断

- 如果没有登录系统，直接访问此CSP文件，会跳转到登录界面。
- 控制当前产品许可下，是否可以访问此界面
- 控制当前安全组下，是否有权限访问此界面

```
<csp:method name=OnPreHTTP arguments="" returnType=%Boolean>
i ##Class(websys.SessionEvents).SessionExpired() q 1
quit 1
</csp:method>
```

3.7.1.2 文件Title写法

统一使用系统的标题头写法，以便统一控制。会显示成：用户名 安全组 科室 医院名 本机IP

```
<TITLE><EXTHEALTH:TRANSLATE id=title>##(%session.Get("TITLE"))##  
</EXTHEALTH:TRANSLATE></TITLE>
```

3.7.1.3 关于head内代码

- 引入 <EXTHEALTH:HEAD></EXTHEALTH:HEAD>

得到统一的js-session对象
得到语音导航功能支持
得到tkMakeServerCall方法支持
得到服务器IP及客户端IP信息
得到客户端操作日志功能
得到websys.js中公共方法

- 引入 <HISUI></HISUI>

得到HISUI开发环境的支持
得到国际化支持
便于统一改变HISUI界面样式
日期格式统一控制
获得统一访问后台的 \$cm, \$m, \$q 方法

- 引用 <ADDINS require="CMgr, CmdShell"></ADDINS>

得到访问客户端能力
尽可能少使用此功能（安全性，稳定性，兼容性）

- csp中head部分代码示例：

```
<HEAD>  
  <EXTHEALTH:HEAD></EXTHEALTH:HEAD>  
  <HISUI></HISUI>  
  <ADDINS require="CmdShell"></ADDINS>  
  <!-- Other Code -->  
</HEAD>
```

3.7.1.4 csp代码模板示例

```
<csp:method name=OnPreHTTP arguments="" returnType=%Boolean>  
  i ##Class(websys.SessionEvents).SessionExpired() q 1  
  quit 1  
</csp:method>  
<HTML>  
  <HEAD>  
    <EXTHEALTH:HEAD></EXTHEALTH:HEAD>  
    <HISUI></HISUI>  
    <ADDINS require="CmdShell"></ADDINS>
```

```
      <!-- Other Code //-->
    </HEAD>
    <BODY>
      <!-- Other Code //-->
    </BODY>
  </HTML>
```